

文章笔记

Claude Code 最佳实践：从”能用”到”真的好用”

基于公开课程资料整理

2026 年 4 月 1 日

原文作者：Mr Panda (PandaTalk8)

发布日期：2026 年 3 月 23 日

来源平台：X (Twitter) 长文

原文链接：<https://x.com/PandaTalk8/status/2035975664730575325>

项目主页：

目录

1	CLAUDE.md：一切的起点	3
1.1	好的 CLAUDE.md 三原则	3
1.2	四级层级与规则拆分	3
1.3	本章小结	3
2	提示词工程：指令的质量决定产出的质量	4
2.1	好指令的三要素	4
2.2	四个官方推荐技巧	4
2.3	本章小结	4
3	Plan Mode：先想清楚再动手	4
3.1	Plan Mode 的正确工作流	5
3.2	何时不需要 Plan Mode	5
3.3	本章小结	5
4	上下文管理：最容易被忽视的关键	5
4.1	上下文里都装了什么	5
4.2	五个实用策略	6
4.3	本章小结	6
5	子 Agent：Claude Code 的分身术	6
5.1	核心价值：隔离上下文	6
5.2	自定义子 Agent	6
5.3	本章小结	7
6	权限与安全：在效率和安全之间找平衡	7
6.1	五种权限模式	7
6.2	权限规则配置	7
6.3	Hooks：让规则变成铁律	7
6.4	本章小结	7
7	Git 工作流与 Worktree	8
7.1	Worktree：隔离的工作空间	8
7.2	Git 铁律	8
7.3	本章小结	8
8	高级功能：Extended Thinking 与 MCP	8
8.1	Extended Thinking	8
8.2	MCP Server	8
8.3	IDE 集成	9
8.4	本章小结	9

9 代码审查与常见陷阱	9
9.1 三个常见问题	9
9.2 Writer/Reviewer 模式	9
9.3 不要用 Claude Code 做的事	9
9.4 本章小结	10
10 快捷键与命令速查	10
11 总结与延伸	10
11.1 拓展阅读	11

1 CLAUDE.md: 一切的起点

什么是 CLAUDE.md ?

CLAUDE.md 是 Claude Code 在每次对话开始时**自动读取**的指令文件，相当于给 AI 助手写的 onboarding 文档——你希望它知道什么，就写什么。不写或随便写几行的后果是：每次对话都要从头解释项目结构、编码规范和技术栈。

1.1 好的 CLAUDE.md 三原则

CLAUDE.md 写作原则

1. **写代码里读不出来的东西**。项目的”为什么”比”是什么”更重要。不需要解释 React 怎么用，但要说明”选 Tailwind 是因为团队统一规范”。
2. **控制在 200 行以内**。官方文档明确提到，CLAUDE.md 太长会导致 Claude 忽略规则。用 Markdown 标题和列表保持可扫描性。
3. **不放频繁变化的内容**。详细的 API 文档、逐文件描述不适合放在里面，放链接就好。

1.2 四级层级与规则拆分

Claude Code 支持四级 CLAUDE.md，按优先级从高到低：

1. **企业级**：由组织管理员统一设定
2. **用户全局**：`~/.claude/CLAUDE.md`
3. **项目级**：项目根目录下的 `CLAUDE.md`
4. **目录级**：子目录中的 `CLAUDE.md`

当项目变大后，可以把规则拆到 `.claude/rules/` 目录下，并用 YAML frontmatter 将规则限定到特定文件模式，这样该规则仅在 Claude 访问匹配的文件时才会加载，节省上下文空间。

1.3 本章小结

CLAUDE.md 是 Claude Code 最被低估也最值得投入的功能。写好一次，每次对话都受益。核心是：写决策理由，不写代码细节；保持精简；善用层级和规则拆分。

2 提示词工程：指令的质量决定产出的质量

2.1 好指令的三要素

目标 + 约束 + 上下文

- **目标**：明确要做什么（“在表格上方加搜索框”）
- **约束**：限定实现方式（“走后端 API，debounce 300ms”）
- **上下文**：引用具体文件、样式规范、现有组件

差的指令：“给用户列表加个搜索功能”。好的指令减少了歧义，Claude 不需要猜你要前端还是后端搜索。

2.2 四个官方推荐技巧

1. **用 @ 引用具体文件**——@src/auth/login.ts 比“那个认证相关的代码”精确得多，还可以指定行号范围 @src/auth/login.ts#5-30。
2. **贴截图和图片**——调试 UI 问题时，Ctrl+V 粘贴“现在长这样”和“我想要这样”的截图，比文字描述有效十倍。
3. **给验证标准**——给 Claude 一个可以自我验证的标准（测试用例、截图、预期输出），它就能自己检查工作质量，减少来回修改。
4. **明确“不要做”的事**——Claude 有时会过度热心，你让它修 bug，它顺手把周围代码也重构了。明确说“只改这个函数，不要动其他代码”。

模糊指令也有用武之地

探索性任务用模糊指令反而更好，比如“这个文件有什么可以改进的地方？”、“帮我理解一下这个模块的架构”。但到了实现阶段，要切回精确模式。

2.3 本章小结

好的提示词 = 目标 + 约束 + 上下文 + 验证标准。探索时用模糊指令，实现时用精确指令。

3 Plan Mode：先想清楚再动手

Plan Mode 是什么？

在 Plan Mode 下，Claude Code 只能读取文件和搜索代码，不能做任何修改。它会探索代码库、分析需求，然后给出一个详细的实现计划。进入方式包括：在提示词中说明“先给我一个计划”、使用 Shift+Tab 切换模式，或在 CLAUDE.md 中配置默认行为。

3.1 Plan Mode 的正确 workflow

官方推荐四步流程：探索 → 规划 → 实现 → 验证

1. 让 Claude 生成初始计划
2. 按 **Ctrl+G** 在编辑器中打开并编辑计划——删掉不同意的部分，补充自己的想法
3. 切回正常模式 (**Shift+Tab**)，让 Claude 按修改后的计划执行
4. 执行完毕后验证结果（跑测试、检查输出）

关键认知：改一个计划只需要一句话，改已经写好的代码要花十倍的时间。

3.2 何时不需要 Plan Mode

判断标准很简单：如果任务的实现方式只有一种（改变量名、修 typo、加一行日志），直接做。如果有多种选择，先规划。

3.3 本章小结

Plan Mode 是复杂任务的必经之路。先规划再动手，可以大幅减少返工。核心操作是 **Ctrl+G** 编辑计划。

4 上下文管理：最容易被忽视的关键

Claude Code 的上下文窗口最大可达 1M token，但它不是无限的，且上下文质量直接决定输出质量。

4.1 上下文里都装了什么

- 对话历史
- Claude 读取的所有文件内容
- 命令执行的输出
- CLAUDE.md 文件（每次都加载）
- Memory 文件（前 200 行）
- 加载的 Skill 和 MCP 工具定义

4.2 五个实用策略

上下文管理五板斧

1. **/clear**——一件事做完就清。不要在同一个对话里又修 bug 又加功能又重构。
2. **/compact**——手动压缩上下文。更好的用法是给压缩加焦点：**/compact** 专注于 API 改动和测试结果。
3. **/context**——查看上下文使用情况，发现哪些 MCP server 占了大量空间。
4. 用子 Agent 隔离噪声——跑测试、分析日志等产生大量输出的操作交给子 Agent，主对话保持干净。
5. 在 **CLAUDE.md** 里写压缩保留指令——告诉 Claude 压缩时必须保留什么关键信息。

Memory 的适用边界

Memory 适合存：偏好、项目约定、历史决策。

Memory 不适合存：代码细节、Git 历史、临时状态——这些从代码和 Git 里获取更准确。

CLAUDE.md 是团队共享的规范；Memory 是个人的记忆。

4.3 本章小结

上下文是稀缺资源。**/clear** 保持对话聚焦，**/compact** 延长对话寿命，子 Agent 隔离噪声输出。

5 子 Agent：Claude Code 的分身术

子 Agent 机制

子 Agent 是独立的 AI 进程，每个有自己的上下文窗口，不会污染主对话。内置三种类型：

- **Explore**：只读，快速搜索代码库，适合找文件、找定义
- **Plan**：只读，研究分析，用于 Plan Mode 下的代码分析
- **General**：完整能力，适合复杂的多步骤任务

5.1 核心价值：隔离上下文

当你让 Claude 跑测试、分析日志、搜索大量文件时，海量输出会塞满上下文窗口。用子 Agent 来做这些事，输出留在子 Agent 的上下文里，只有摘要返回给你。

按 **Ctrl+B** 可以把子 Agent 放到后台运行——适合跑测试套件、大范围代码搜索、不紧急的代码审查。

5.2 自定义子 Agent

在 **.claude/agents/** 目录下创建自定义 Agent 的 Markdown 文件，然后在对话中用 **@"agent-name (agent)"** 调用。可以为特定场景（如 code review）定制专属 Agent。

5.3 本章小结

子 Agent 的核心价值是上下文隔离。把高噪声操作（测试、日志分析、大范围搜索）交给子 Agent，保持主对话干净高效。

6 权限与安全：在效率和安全之间找平衡

6.1 五种权限模式

Claude Code 提供五种权限模式，用 **Shift+Tab** 在对话中循环切换，从全自动到完全手动确认，覆盖不同的安全需求场景。

6.2 权限规则配置

在 `.claude/settings.json` 里精细控制每种工具的权限。

权限优先级：deny > ask > allow

可以大胆地 allow 常用命令，同时用 deny 保护敏感文件。比如 `.env` 文件无论如何都不应该被编辑。

设置文件也有多级层次：团队共享规则放 `.claude/settings.json`，个人偏好放 `~/.claude/settings.json`，敏感配置放 `.claude/settings.local.json`（加到 `.gitignore`）。

6.3 Hooks：让规则变成铁律

CLAUDE.md 是”建议”，Hooks 是”铁律”

CLAUDE.md 里的指令 Claude 大部分时候会遵守，但偶尔会忘。Hooks 是在特定生命周期事件上自动触发的 shell 命令，无论如何都会执行。配置在 `settings.json` 里。

Hooks 的关键事件类型包括编辑文件前/后、执行命令前/后、上下文压缩后等。Hook 命令返回 `exit code 0` 表示允许，`exit code 2` 表示阻止，因此可以写任意复杂的判断逻辑。

实用场景举例：

- **自动格式化**：每次编辑后运行 Prettier
- **保护关键文件**：阻止修改生产配置
- **压缩后重注入**：对话被压缩后自动重新注入关键指令，解决”长对话压缩丢信息”的痛点

Hook 有四种类型：`command`（shell 命令）、`http`（POST 请求）、`prompt`（单次 LLM 判断）、`agent`（启动子 Agent 验证）。

6.4 本章小结

权限管理的核心是分层：deny 保护底线，allow 提升效率。Hooks 把 CLAUDE.md 中的”建议”变成可执行的”铁律”，是保障代码安全的最后一道防线。

7 Git 工作流与 Worktree

7.1 Worktree: 隔离的工作空间

Git Worktree 的价值

Worktree 会在 `.claude/worktrees/` 下创建一个独立的 Git 分支副本。Claude 在里面怎么折腾都不影响主分支。退出时：没有改动则自动清理，有改动则提示保留或删除。对探索性任务特别有用——不确定某个重构方案能不能行？开个 worktree 试试，不行就扔掉，零风险。

7.2 Git 铁律

四条不可违反的 Git 规则

1. **永远不要让 Claude Code 自动 push**——它可以 commit，但 push 必须由你确认。一旦 push 到远端，回退成本就大了。
2. **频繁 commit**——做完一个小功能就 commit。用 `/rewind` 回退的前提是有 commit 记录。
3. **警惕破坏性操作**——`git reset --hard`、`git push --force`、`rm -rf` 没有 undo，可以在权限规则里直接 deny。
4. **从 PR 恢复上下文**——自动加载 PR 的改动和讨论作为上下文，适合 code review 或继续别人的工作。

7.3 本章小结

Worktree 提供了零风险的实验环境。Git 操作的核心原则是：频繁 commit、禁止自动 push、deny 破坏性命令。

8 高级功能：Extended Thinking 与 MCP

8.1 Extended Thinking

对于复杂问题，可以开启 Extended Thinking 模式让 Claude 在回答前花更多时间推理。

何时使用 Extended Thinking

适合：复杂架构决策、棘手的多因素 debug、多步骤重构方案、多方案利弊权衡。

不需要：简单代码修改、格式化/重命名、已经很明确的 bug 修复。

小技巧：在提示词里写 `ultrathink` 可以强制触发最高推理深度。

8.2 MCP Server

MCP (Model Context Protocol) 让 Claude Code 能和外部工具通信——GitHub、数据库、Slack、Notion 等。配置在项目根目录的 `.mcp.json` 里。

MCP 的上下文开销

每个 MCP server 会额外消耗 100–500+ token 的工具定义。用 `/mcp` 命令查看每个 server 的开销，禁用不用的 server。**不要贪多**——常用的 2–3 个就够了，装太多反而影响代码分析质量。

8.3 IDE 集成

Claude Code 不仅可以在终端使用，还有 VS Code 扩展和 JetBrains 插件：

- **VS Code**：并排 diff 视图、`Option+K` 快速引用文件、多标签对话、`Cmd+Shift+Esc` 新建对话
- **JetBrains**：IntelliJ/PyCharm/WebStorm 等均支持，在 Plugins 市场搜索安装

8.4 本章小结

Extended Thinking 适合复杂推理任务。MCP 扩展了 Claude 的能力边界，但要注意上下文开销。IDE 集成让 workflow 更顺滑。

9 代码审查与常见陷阱

9.1 三个常见问题

Claude Code 代码的典型问题

1. **过度工程**——你要一个简单工具函数，它给你搞出完整的抽象层（泛型 + 接口 + 工厂模式）。**解法**：在 CLAUDE.md 写”优先用简单方案”，或在指令里明确”不需要抽象”。
2. **幻觉 API**——使用不存在的 API 或过时的语法，尤其是小众库的新版本。**解法**：跑测试，指令里包含验证标准。
3. **改动范围膨胀**——你让它改一个函数，它把整个文件都重构了。**解法**：明确说”只改 X，不要动其他代码”，或用 `/freeze` 限制编辑范围。

9.2 Writer/Reviewer 模式

官方推荐的高级模式：用两个独立会话分别扮演”写代码”和”审查代码”的角色。两个会话有独立上下文，Reviewer 不受 Writer 思路影响，能发现 Writer 的盲点。

9.3 不要用 Claude Code 做的事

四条红线

1. **不要让它做你完全不了解的事**——你无法审查你不懂的东西。
2. **不要在无版本控制的环境下用**——没有 Git 就没有回退能力，等于裸奔。
3. **不要一口气扔一个巨大的任务**——拆成小步骤，每步确认后再做下一步。
4. **不要指望一次就完美**——迭代是正常的，用 `/rewind` 回退，用精确反馈描述”哪里不对”。

9.4 本章小结

信任但要验证。用 CLAUDE.md 约束过度工程，用测试拦截幻觉 API，用明确指令控制改动范围。Writer/Reviewer 双会话模式适合重要功能开发。

10 快捷键与命令速查

表 1: 最常用的快捷键

快捷键	功能
Shift+Tab	切换权限模式 / 切换 Plan Mode
Ctrl+G	在编辑器中打开并编辑计划
Ctrl+B	将子 Agent 放到后台运行
Ctrl+V	粘贴图片到对话
Option+K / Alt+K	(VS Code) 快速插入 @ 文件引用
Cmd+Shift+Esc	(VS Code) 打开新对话标签

表 2: 最常用的斜杠命令

命令	功能
/clear	清空上下文，保留 CLAUDE.md
/compact	压缩上下文，可加焦点参数
/context	查看上下文使用情况
/memory	查看和管理 Memory
/mcp	查看 MCP server 及其开销
/init	自动分析代码库生成 CLAUDE.md 初稿
/rewind	回退到任意 checkpoint
/freeze	限制编辑范围到指定目录

11 总结与延伸

原文作者 Mr Panda 在大半年的 Claude Code 实战中提炼出一套清晰的使用哲学：

Claude Code 核心使用哲学

它是一个极其能干的协作者，但不是自动驾驶。方向盘始终在你手里。工具的价值，永远取决于使用它的人。

按重要性排序的六条建议：

- 1. 写好 CLAUDE.md——一次投入，每次对话都受益
- 2. 给精确的指令——目标 + 约束 + 验证标准

3. 用 **Plan Mode**——复杂任务先规划再动手
4. 管理上下文——`/clear`、`/compact`、子 Agent
5. 控制权限——`deny` 危险操作，`allow` 常用命令
6. 频繁 **commit**——保留回退能力

11.1 拓展阅读

- Claude Code 官方文档: <https://docs.anthropic.com/en/docs/claude-code>
- Model Context Protocol 规范: <https://modelcontextprotocol.io>
- 原文链接: <https://x.com/PandaTalk8/status/2035975664730575325>